WEEK 15

Final Project, Part 1: Build the Expense Tracker

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

These last two weeks you build **one complete app** by yourself, from an empty project to a release on GitHub (and the Play Store if you like). It uses everything from the course: Compose, MVVM, Room, Retrofit, Hilt, DataStore and tests. This is the app you'll show in interviews, so take your time and make it yours.

This PDF builds an **Expense Tracker** step by step. If you'd prefer a Recipe or News app, follow the same steps: the layers, the patterns and most of the code stay the same, only the data changes.

Day	Topic	You will be able to...
Day 1	Plan the project	Write a feature list, sketch screens, model data, set up Git
Day 2	The domain layer	Handle money safely, validate input, write use cases
Day 3	The data layer	Save expenses with Room and fetch exchange rates with Retrofit
Day 4	Wiring with Hilt	Connect all the layers with Hilt modules
Day 5	The expense list screen	Show expenses grouped by day, with delete and undo
Day 6	Add and edit screen	Build a validated form with a date picker, and navigation

Tip: All the plain Kotlin code (money, validation, summaries, use cases, JSON) was **run**, and the outputs are real. Room, Retrofit, Hilt and Navigation couldn't be downloaded here, so the Android code was compile-checked with the real Compose compiler against stand-ins that copy those libraries' names and shapes. Always run the app on a phone or emulator.

WEEK 15 • DAY 1

Plan Your Final Project

Today's goal: Choose the app, write the feature list, sketch the screens, model the data and set up the project with Git.

1. Why plan first?

In a team, nobody starts coding on day one. First the team agrees **what** to build (features), **how it looks** (screens) and **how it's organised** (architecture). An hour of planning saves days of rewriting. You'll do the same, on a small scale.

2. Choose your app

App	Main data	API (Retrofit)	Good because...
Expense Tracker (this PDF)	Expenses saved in Room	Exchange rates	Useful every day; lots of logic to test
Recipe App	Favourite recipes in Room	TheMealDB	Images, search and favourites
News App	Saved articles in Room	A news API (needs a free key)	Offline-first reading

3. Write the feature list (MVP)

MVP means Minimum Viable Product: the smallest version that is actually useful. Sort every idea into three groups and build the **Must** group first. Only add the others when the Must features work.

Priority	Feature
Must	Add, edit and delete an expense (title, amount, category, date, note)
Must	List of expenses, newest first, grouped by day, with this month's total
Must	Data stays after closing the app (Room)
Should	Monthly summary by category, with bars
Should	Monthly budget with a warning (DataStore)
Should	Show a month's total in another currency (Retrofit)
Could	Dark mode setting, export to CSV, reminders (Week 13)

Tip: Write your features as **user stories**: *"As a student, I want to see how much I spent on food this month, so that I can save money."* Each story becomes a GitHub issue, and each issue becomes a branch and a commit.

4. Sketch the screens

Sketch: Expense list (home)

```
+--------------------------------+
| Expenses                       |
+--------------------------------+
|  This month                    |
|  Rs. 4,320.00                  |
|                                |
|  Today              Rs. 400.00 |
|  (F) Lunch          Rs. 120.00 |
|  (T) Auto           Rs. 280.00 |
|  Yesterday          Rs. 399.00 |
|  (B) Phone bill     Rs. 399.00 |
|                           [ + ] |
+--------------------------------+
| Expenses | Summary | Settings  |
+--------------------------------+
```

Sketch: Add / edit expense

```
+--------------------------------+
| <-  New expense                |
+--------------------------------+
|  [ Title               ]       |
|  [ Amount (Rs.)        ]       |
|  Category                      |
|  (Food) (Travel) (Bills) ...   |
|  [ Date: 2025-03-10    ]       |
|  [ Note (optional)     ]       |
|                                |
|  [          Save          ]    |
+--------------------------------+
```

Draw these on paper first (or in Figma, if your team uses it). Sketches show which **state** each screen needs: the list screen needs sections and a total; the form needs each field's text and its error.

5. Model the data

The data model (run)

```
import java.time.LocalDate

enum class Category(val label: String) {
    FOOD("Food"), TRAVEL("Travel"), SHOPPING("Shopping"), BILLS("Bills"), HEALTH("Health"), OTHER("Other")
}

data class Expense(
    val id: Long = 0,                // 0 means "not saved yet"
    val title: String,
    val amountPaise: Long,           // Rs. 12.50 is stored as 1250
    val category: Category,
    val date: LocalDate,
    val note: String = "",
)

fun main() {
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, LocalDate.of(2025, 3, 3)),
        Expense(2, "Bus pass", 50000, Category.TRAVEL, LocalDate.of(2025, 3, 1)),
        Expense(3, "Phone bill", 39900, Category.BILLS, LocalDate.of(2025, 3, 5)),
    )
    for (e in expenses.sortedByDescending { it.date }) {
        println("${e.date}  ${e.title.padEnd(11)}${e.category.label.padEnd(8)}Rs. ${e.amountPaise / 100}")
    }
}
```

Output

```
2025-03-05  Phone bill Bills   Rs. 399
2025-03-03  Lunch      Food    Rs. 120
2025-03-01  Bus pass   Travel  Rs. 500
```

Choice	Why
amountPaise: Long	Money is stored in paise (whole numbers), never as Double. Tomorrow you'll see why
category: Category (enum)	Only valid categories are possible; no typos like "fod"
date: LocalDate	A date without a time. Easy to compare, group by day and by month
id: Long = 0	Room fills in the real id when the expense is saved

Careful: java.time (LocalDate, YearMonth) needs **minSdk 26** or newer. That's the default for new projects in Android Studio, so you're fine. (Older apps turn on "core library desugaring" to use it.)

6. Plan the architecture

Sketch: Layers of the Expense Tracker

```
ui/       ExpenseListScreen   AddEditScreen   SummaryScreen   SettingsScreen
             |  ViewModels (StateFlow<UiState>)
             v
domain/   Expense, Category, validate(), summarize(), SaveExpenseUseCase
             |  ExpenseRepository (interface)
             v
data/     OfflineExpenseRepository -> Room (ExpenseDao)
          NetworkCurrencyRepository -> Retrofit (CurrencyApi)
          SettingsRepository -> DataStore
di/       Hilt modules that connect everything
```

Package	What goes in it
com.anjana.expensetracker.domain	Models, money helpers, validation, use cases, repository interfaces
...data.local	Room entity, DAO, database, OfflineExpenseRepository
...data.remote	Retrofit API, DTOs, NetworkCurrencyRepository
...di	Hilt modules
...ui.list, ...ui.addedit, ...ui.summary, ...ui.settings	One package per screen: Screen + ViewModel + UiState
...ui.theme	Colours, typography, ExpenseTheme

7. Create the project and the Git repository

- Android Studio → **New Project** → **Empty Activity**. Name: **Expense Tracker**, package: **com.anjana.expensetracker**, minimum SDK: **API 26**.
- Run it once on the emulator to check the empty app works.
- On GitHub, create a new **empty** repository called **expense-tracker** (no README yet; you'll write one in Week 16).
- Android Studio already made a good **.gitignore**. Check that it contains **local.properties** and **/build**.

First commit and push (Terminal in Android Studio)

```
git init
git add .
git commit -m "Create empty Expense Tracker project"
git branch -M main
git remote add origin https://github.com/<your-username>/expense-tracker.git
git push -u origin main

# For each feature: a branch, small commits, then merge
git checkout -b feature/domain-layer
git commit -m "Add Expense model and money helpers"
git checkout main
git merge feature/domain-layer
git push
```

Good commit message	Bad commit message
Add validation for expense form	changes
Fix total not updating after delete	fixed bug
Show monthly summary bars	final final 2

8. Your 12-day schedule

Week 15	Week 16
Day 1: plan and set up	Day 1: summary screen and bottom navigation
Day 2: domain layer	Day 2: settings, budget and polish
Day 3: data layer (Room and Retrofit)	Day 3: test the project
Day 4: Hilt wiring	Day 4: release build and signing
Day 5: expense list screen	Day 5: Play Store basics
Day 6: add/edit screen and navigation	Day 6: README, portfolio and next steps

Practice Exercises

1. Add a **PaymentMethod** enum (CASH, UPI, CARD) with a label, and add a **paymentMethod** property to Expense with the default UPI. Create one expense paid by cash and print it.

2. Write an extension function **List<Expense>.totalFor(category: Category): Long** and print the food total for a small list.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What does MVP mean, and why build it first?

Q2. Why is category an enum instead of a String?

Q3. Why is the default id 0?

Q4. What minSdk do you need for java.time without extra setup?

WEEK 15 • DAY 2

The Domain Layer: Money, Rules and Use Cases

Today's goal: Write the pure Kotlin heart of the app: money helpers, form validation, the monthly summary and a use case.

1. Start with the domain

The **domain layer** is plain Kotlin: no Android, no Room, no Compose. It holds the app's **rules**. Because it has no Android parts, you can run and test it in seconds, and it never needs to change when the UI changes. Build it first.

2. Never use Double for money

Why Double is dangerous for money (run)

```
fun main() {
    println(0.1 + 0.2)

    var total = 0.0
    repeat(10) { total += 0.1 }          // add 10 paise... I mean 0.1 rupees, ten times
    println(total)
    println(total == 1.0)

    var paise = 0L
    repeat(10) { paise += 10 }           // 10 paise, ten times
    println(paise == 100L)
}
```

Output

```
0.30000000000000004
0.9999999999999999
false
true
```

Double stores numbers in binary, and 0.1 can't be stored exactly, so small errors creep in. For money, store whole **paise** in a **Long**: Rs. 12.50 becomes 1250. Convert only when showing it to the user. Banks and payment apps do the same.

3. Money helpers

Money.kt (domain)

```
fun formatRupees(paise: Long): String {
    val sign = if (paise < 0) "-" else ""
    val abs = kotlin.math.abs(paise)
    return sign + "Rs. " + "%,d".format(abs / 100) + "." + (abs % 100).toString().padStart(2, '0')
}

// For putting a saved amount back into a text field: 125050 -> "1250.50"
fun toAmountText(paise: Long): String = "${paise / 100}." + (paise % 100).toString().padStart(2, '0')

// "120" -> 12000, "99.5" -> 9950, "12.345" -> null, "abc" -> null
fun parseRupees(text: String): Long? {
    val value = text.trim().replace(",", "").toBigDecimalOrNull() ?: return null
    if (value.scale() > 2) return null                // more than 2 decimal places
    return value.movePointRight(2).toLong()
}

fun main() {
    println(formatRupees(125050))
    println(formatRupees(99))
    println(parseRupees("120"))
    println(parseRupees(" 99.5 "))
    println(parseRupees("12.345"))
    println(parseRupees("abc"))
    println(toAmountText(125050))
}
```

Output

```
Rs. 1,250.50
Rs. 0.99
12000
9950
null
null
1250.50
```

Piece	What it does
"%,d".format(n)	Adds thousands separators: 1250 → "1,250"
padStart(2, '0')	5 paise → "05", so Rs. 1.05 shows correctly
toBigDecimalOrNull()	Turns text into an exact decimal number, or null if it isn't a number
scale() > 2	More than two digits after the point, like 12.345, isn't valid money
movePointRight(2)	Moves the decimal point: 99.50 → 9950 (paise)

4. Validating the form

The form gives us **text** (whatever the user typed). Validation checks it and returns either **Valid** or a map of **errors per field**, so the screen can show each message under the right text field.

Validation.kt (domain)

```
enum class Field { TITLE, AMOUNT, DATE }

data class ExpenseForm(
    val title: String,
    val amountText: String,          // exactly what the user typed
    val category: Category,
    val date: LocalDate,
    val note: String = "",
```

```
)

sealed interface ValidationResult {
    data object Valid : ValidationResult
    data class Invalid(val errors: Map<Field, String>) : ValidationResult
}

const val MAX_AMOUNT_PAISE = 100_000_000L            // Rs. 10,00,000 (10 lakh)

fun validate(form: ExpenseForm, today: LocalDate): ValidationResult {
    val errors = mutableMapOf<Field, String>()

    val title = form.title.trim()
    if (title.isEmpty()) errors[Field.TITLE] = "Enter a title"
    else if (title.length > 40) errors[Field.TITLE] = "Keep the title under 40 characters"

    val paise = parseRupees(form.amountText)
    when {
        form.amountText.isBlank() -> errors[Field.AMOUNT] = "Enter an amount"
        paise == null -> errors[Field.AMOUNT] = "Enter a number like 120 or 99.50"
        paise <= 0 -> errors[Field.AMOUNT] = "Amount must be more than zero"
        paise > MAX_AMOUNT_PAISE -> errors[Field.AMOUNT] = "That amount is too large"
    }

    if (form.date.isAfter(today)) errors[Field.DATE] = "Date can't be in the future"

    return if (errors.isEmpty()) ValidationResult.Valid else ValidationResult.Invalid(errors)
}

fun main() {
    val today = LocalDate.of(2025, 3, 10)
    println(validate(ExpenseForm("Lunch", "120", Category.FOOD, today), today))

    val bad = ExpenseForm("  ", "12.345", Category.FOOD, today.plusDays(1))
    val result = validate(bad, today)
    if (result is ValidationResult.Invalid) {
        result.errors.forEach { (field, message) -> println("$field: $message") }
    }
}
```

Output

```
Valid
TITLE: Enter a title
AMOUNT: Enter a number like 120 or 99.50
DATE: Date can't be in the future
```

Tip: today is a parameter instead of calling LocalDate.now() inside validate(). That makes the function easy to test: a test can pick any date. You'll see this idea again in the use case.

5. The monthly summary

Summary.kt (domain)

```
import java.time.YearMonth

data class CategoryTotal(val category: Category, val totalPaise: Long, val percent: Int)

data class MonthSummary(
    val month: YearMonth,
    val totalPaise: Long,
    val count: Int,
    val byCategory: List<CategoryTotal>,       // biggest first
)

fun summarize(expenses: List<Expense>, month: YearMonth): MonthSummary {
    val inMonth = expenses.filter { YearMonth.from(it.date) == month }
    val total = inMonth.sumOf { it.amountPaise }
    val byCategory = inMonth
        .groupBy { it.category }
        .map { (category, list) ->
            val sum = list.sumOf { it.amountPaise }
            CategoryTotal(category, sum, if (total == 0L) 0 else (sum * 100 / total).toInt())
        }
        .sortedByDescending { it.totalPaise }
    return MonthSummary(month, total, inMonth.size, byCategory)
}

fun main() {
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, LocalDate.of(2025, 3, 3)),
        Expense(2, "Bus pass", 50000, Category.TRAVEL, LocalDate.of(2025, 3, 1)),
        Expense(3, "Dinner", 28000, Category.FOOD, LocalDate.of(2025, 3, 8)),
        Expense(4, "Phone bill", 39900, Category.BILLS, LocalDate.of(2025, 3, 5)),
        Expense(5, "Coffee", 5000, Category.FOOD, LocalDate.of(2025, 2, 27)),     // February: not counted
    )
    val s = summarize(expenses, YearMonth.of(2025, 3))
    println("${s.month}: ${formatRupees(s.totalPaise)} in ${s.count} expenses")
    for (c in s.byCategory) {
        println("  ${c.category.label.padEnd(7)} ${formatRupees(c.totalPaise).padStart(12)}  ${c.percent}%")
    }
}
```

Output

```
2025-03: Rs. 1,299.00 in 4 expenses
  Travel    Rs. 500.00  38%
  Food      Rs. 400.00  30%
  Bills     Rs. 399.00  30%
```

The percentages add up to 98%, not 100%, because integer division rounds down (30.79% becomes 30%). That's fine for a chart. Small details like this are good things to mention in an interview.

6. The repository interface and a use case

The domain layer only knows an **interface** for storage. Room comes tomorrow; until then an in-memory version lets you run everything now. The **use case** combines validation and saving in one place, so the ViewModel stays small.

Repository.kt and SaveExpenseUseCase.kt (domain)

```
import kotlinx.coroutines.flow.Flow

interface ExpenseRepository {
    fun observeExpenses(): Flow<List<Expense>>        // newest first
    suspend fun getExpense(id: Long): Expense?
    suspend fun upsert(expense: Expense): Long         // insert or update; returns the id
    suspend fun delete(id: Long)
```

```
}

// "What is today's date?" behind an interface, so tests can fix the date.
fun interface TodayProvider {
    fun today(): LocalDate
}

import javax.inject.Inject

class SaveExpenseUseCase @Inject constructor(
    private val repository: ExpenseRepository,
    private val todayProvider: TodayProvider,
) {
    suspend operator fun invoke(id: Long, form: ExpenseForm): ValidationResult {
        val result = validate(form, todayProvider.today())
        if (result is ValidationResult.Valid) {
            val expense = Expense(
                id = id,
                title = form.title.trim(),
                amountPaise = parseRupees(form.amountText)!!,   // safe: validate() checked it
                category = form.category,
                date = form.date,
                note = form.note.trim(),
            )
            repository.upsert(expense)
        }
        return result
    }
}

import kotlinx.coroutines.flow.*

// A simple repository that keeps everything in memory.
// Useful before Room is ready, and later as a fake in tests.
class InMemoryExpenseRepository : ExpenseRepository {
    private val items = MutableStateFlow<List<Expense>>(emptyList())
    private var nextId = 1L

    override fun observeExpenses(): Flow<List<Expense>> =
        items.map { list -> list.sortedWith(compareByDescending<Expense> { it.date }.thenByDescending { it.id }) }

    override suspend fun getExpense(id: Long): Expense? = items.value.find { it.id == id }

    override suspend fun upsert(expense: Expense): Long {
        val id = if (expense.id == 0L) nextId++ else expense.id
        items.update { list -> list.filterNot { it.id == id } + expense.copy(id = id) }
        return id
    }

    override suspend fun delete(id: Long) {
        items.update { list -> list.filterNot { it.id == id } }
    }
}

import kotlinx.coroutines.flow.first
import kotlinx.coroutines.runBlocking

fun main() = runBlocking {
    val repository = InMemoryExpenseRepository()
    val save = SaveExpenseUseCase(repository) { LocalDate.of(2025, 3, 10) }    // today is fixed

    println(save(0, ExpenseForm("Lunch", "120", Category.FOOD, LocalDate.of(2025, 3, 10))))
    println(save(0, ExpenseForm("", "50", Category.FOOD, LocalDate.of(2025, 3, 10))))
    println(save(0, ExpenseForm("Bus pass", "500", Category.TRAVEL, LocalDate.of(2025, 3, 1))))

    repository.observeExpenses().first().forEach { println("${it.id} ${it.title} ${formatRupees(it.amountPaise)}") }
}
```

Output

```
Valid
Invalid(errors={TITLE=Enter a title})
```

```
Valid
1 Lunch Rs. 120.00
2 Bus pass Rs. 500.00
```

Piece	Why
fun interface TodayProvider	"Today" can be replaced in tests; the lambda { LocalDate.of(...) } creates one
operator fun invoke	Lets you call the use case like a function: save(0, form)
@Inject constructor	Hilt will create the use case for you (Day 4). It does nothing in plain Kotlin
InMemoryExpenseRepository	Lets you build and test before Room is ready; reused as a fake in Week 16 tests

Commit your work

```
git add .
git commit -m "Add domain layer: money, validation, summary and save use case"
git push
```

Practice Exercises

1. Add a rule to validate(): the note must be at most **100 characters**, with the error "Note is too long". You'll need a new **NOTE** value in Field. Test it with a 120-character note.

2. Write **fun averagePerDay(expenses: List<Expense>, month: YearMonth): Long** that returns the month's total divided by the number of days in that month (use **month.lengthOfMonth()**). Print it for March 2025 using the summary example's list.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why store money as Long paise instead of Double rupees?

Q2. What does parseRupees("12.345") return, and why?

Q3. Why does validate() take today as a parameter?

Q4. What does the domain layer depend on?

WEEK 15 • DAY 3

The Data Layer: Room and Retrofit

Today's goal: Store expenses with Room, map between database and domain models, and fetch exchange rates with Retrofit.

1. What the data layer does

Data	Where it lives	Library
Expenses	On the phone, in a database	Room (Week 11)
Exchange rates	On the internet	Retrofit (Week 10)
Settings (budget, theme)	On the phone, small key-values	DataStore (Week 16 Day 2)

The data layer **implements** the repository interface from yesterday. The rest of the app never knows whether the data came from Room, the network or memory.

app/build.gradle.kts dependencies for the whole project (Android only, versions as in your project)

```
plugins {
    id("com.google.devtools.ksp")
    id("com.google.dagger.hilt.android")
    alias(libs.plugins.kotlin.serialization)
}

dependencies {
    // Room
    implementation("androidx.room:room-runtime:2.6.1")
    implementation("androidx.room:room-ktx:2.6.1")
    ksp("androidx.room:room-compiler:2.6.1")
    // Retrofit + JSON
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-kotlinx-serialization:2.11.0")
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")
    // Hilt
    implementation("com.google.dagger:hilt-android:2.52")
    ksp("com.google.dagger:hilt-compiler:2.52")
    implementation("androidx.hilt:hilt-navigation-compose:1.2.0")
    // Navigation, lifecycle, DataStore
    implementation("androidx.navigation:navigation-compose:2.8.4")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.7")
    implementation("androidx.datastore:datastore-preferences:1.1.1")
}
```

Careful: If Gradle complains about versions (like the AAR metadata error you had earlier), let Android Studio suggest the newest versions that match your **compileSdk** and Kotlin version, or copy the versions from a working project in your team.

2. The Room entity and mappers

The database has its own model, **ExpenseEntity**, with simple column types. **Mapper functions** convert between the entity and the domain Expense. This keeps Room's annotations out of the domain layer.

ExpenseEntity.kt (data.local)

```
@Entity(tableName = "expenses")
data class ExpenseEntity(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val title: String,
    @ColumnInfo(name = "amount_paise") val amountPaise: Long,
    val category: String,                                  // the enum's name, e.g. "FOOD"
    @ColumnInfo(name = "epoch_day") val epochDay: Long,  // days since 1 Jan 1970
    val note: String,
)

fun ExpenseEntity.toDomain() = Expense(
    id = id,
    title = title,
    amountPaise = amountPaise,
    category = Category.valueOf(category),
    date = LocalDate.ofEpochDay(epochDay),
    note = note,
)

fun Expense.toEntity() = ExpenseEntity(
    id = id,
    title = title,
    amountPaise = amountPaise,
    category = category.name,
    epochDay = date.toEpochDay(),
    note = note,
)
```

What the mappers store (run)

```
import java.time.LocalDate

enum class Category { FOOD, TRAVEL, BILLS }

fun main() {
    val date = LocalDate.of(2025, 3, 10)
    val stored = date.toEpochDay()             // what goes into the epoch_day column
    println(stored)
    println(LocalDate.ofEpochDay(stored))      // and back again

    val name = Category.TRAVEL.name            // what goes into the category column
    println(name)
    println(Category.valueOf(name) == Category.TRAVEL)
}
```

Output

```
20157
2025-03-10
TRAVEL
true
```

Careful: The category is saved by its **name**. If you rename FOOD to FOOD_AND_DRINK later, **Category.valueOf("FOOD")** crashes for old rows. Never rename enum values that are already stored; add new ones instead (or write a migration).

3. The DAO and the database

ExpenseDao.kt and ExpenseDatabase.kt (data.local)

```
@Dao
interface ExpenseDao {
    @Query("SELECT * FROM expenses ORDER BY epoch_day DESC, id DESC")
    fun observeAll(): Flow<List<ExpenseEntity>>

    @Query("SELECT * FROM expenses WHERE id = :id")
    suspend fun getById(id: Long): ExpenseEntity?

    @Upsert                                     // insert, or update if the id already exists
    suspend fun upsert(entity: ExpenseEntity): Long

    @Query("DELETE FROM expenses WHERE id = :id")
    suspend fun deleteById(id: Long)
}

@Database(entities = [ExpenseEntity::class], version = 1, exportSchema = false)
abstract class ExpenseDatabase : RoomDatabase() {
    abstract fun expenseDao(): ExpenseDao
}
```

Function	Why it looks like this
observeAll(): Flow	Room emits a new list every time the table changes, so the UI updates by itself
ORDER BY epoch_day DESC, id DESC	Newest day first; within a day, the newest expense first
@Upsert	One function for add and edit. Returns the new row id, or -1 when it updated an existing row
deleteById(id)	Deleting by id is simpler than passing the whole entity

4. The Room repository

OfflineExpenseRepository.kt (data.local)

```
class OfflineExpenseRepository @Inject constructor(
    private val dao: ExpenseDao,
) : ExpenseRepository {

    override fun observeExpenses(): Flow<List<Expense>> =
        dao.observeAll().map { list -> list.map { it.toDomain() } }

    override suspend fun getExpense(id: Long): Expense? = dao.getById(id)?.toDomain()

    override suspend fun upsert(expense: Expense): Long {
        val rowId = dao.upsert(expense.toEntity())
        return if (expense.id == 0L) rowId else expense.id   // @Upsert returns -1 for an update
    }

    override suspend fun delete(id: Long) = dao.deleteById(id)
}
```

Compare it with the InMemoryExpenseRepository: same interface, different storage. The ViewModels won't change at all when you switch from one to the other on Day 4.

5. Exchange rates with Retrofit

For the "show in another currency" feature you'll use the free **Frankfurter** API, which needs no key. A request like **https://api.frankfurter.app/latest?from=INR&to=USD** returns JSON like this:

Decoding the API's JSON (run, with example numbers)

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class RatesDto(
    val amount: Double,
    val base: String,
    val date: String,
    val rates: Map<String, Double>,     // e.g. {"USD": 0.0115}
)

fun main() {
    val json = """{"amount":1.0,"base":"INR","date":"2025-03-07","rates":{"USD":0.01148}}"""
    val dto = Json { ignoreUnknownKeys = true }.decodeFromString<RatesDto>(json)

    val totalPaise = 1_250_000L                       // Rs. 12,500.00
    val usd = totalPaise / 100.0 * dto.rates.getValue("USD")
    println("Rate on ${dto.date}: ${dto.rates["USD"]}")
    println("Rs. 12,500.00 is about US$ " + "%.2f".format(usd))
}
```

Output

```
Rate on 2025-03-07: 0.01148
Rs. 12,500.00 is about US$ 143.50
```

Tip: Using Double here is fine: a converted amount is only an **estimate** for display. You never store it or add it up. Before you code, open the URL in your browser to check the API still answers the same way (see frankfurter.dev for its docs).

CurrencyApi.kt and NetworkCurrencyRepository.kt (data.remote)

```
@Serializable
data class RatesDto(
    val amount: Double,
    val base: String,
    val date: String,
    val rates: Map<String, Double>,     // e.g. {"USD": 0.0115}
)

interface CurrencyApi {
    // GET https://api.frankfurter.app/latest?from=INR&to=USD
    @GET("latest")
    suspend fun latest(@Query("from") from: String = "INR", @Query("to") to: String): RatesDto
}

interface CurrencyRepository {
    suspend fun rateFromInr(to: String): Result<Double>
}

class NetworkCurrencyRepository @Inject constructor(
    private val api: CurrencyApi,
) : CurrencyRepository {
    override suspend fun rateFromInr(to: String): Result<Double> =
        try {
            val rate = api.latest(to = to).rates[to]
            if (rate != null) Result.success(rate) else Result.failure(IllegalStateException("No rate for $to"))
        } catch (e: IOException) {
            Result.failure(e)                // no internet, timeout
        } catch (e: HttpException) {
            Result.failure(e)                // server answered with 4xx/5xx
        }
}
```

Result<Double> is Kotlin's built-in type for "a value or an error". The ViewModel will use **result.fold(onSuccess = ..., onFailure = ...)** to show the converted amount or an error message.

AndroidManifest.xml: don't forget the internet permission

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />
    <application ...>
        ...
    </application>
</manifest>
```

Practice Exercises

1. Add a DAO function **observeBetween(startDay: Long, endDay: Long): Flow<List<ExpenseEntity>>** that returns the expenses whose epoch_day is between the two values (inclusive), newest first.
2. Write **fun RatesDto.convert(paise: Long, to: String): Double?** that returns the converted amount, or null if the currency isn't in rates. Try it with the JSON above for "USD" and "EUR".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why have a separate ExpenseEntity instead of putting @Entity on Expense?

Q2. How is a LocalDate stored in Room here?

Q3. What does @Upsert return when it updates an existing row?

Q4. Why does the currency repository return Result?

WEEK 15 • DAY 4

Wiring Everything with Hilt

Today's goal: Connect the domain, data and UI layers with Hilt modules, so every class gets what it needs automatically.

1. Who needs what?

Before writing modules, draw the **dependency graph**: an arrow means "needs". Hilt will build this graph for you.

Sketch: The Expense Tracker's dependency graph

```
ExpenseListViewModel --> ExpenseRepository --> ExpenseDao --> ExpenseDatabase --> Context
                     --> TodayProvider
AddEditViewModel     --> SaveExpenseUseCase --> ExpenseRepository, TodayProvider
SummaryViewModel     --> ExpenseRepository, CurrencyRepository --> CurrencyApi --> Retrofit --> Json
```

Class	How Hilt gets it
SaveExpenseUseCase, OfflineExpenseRepository, NetworkCurrencyRepository	@Inject constructor: Hilt can build these itself
ExpenseRepository, CurrencyRepository (interfaces)	@Binds: "when someone asks for the interface, give this class"
ExpenseDatabase, ExpenseDao, Retrofit, CurrencyApi, Json	@Provides: built by a library's builder, so you write the code
TodayProvider	@Provides: a small lambda returning LocalDate.now()

2. Gradle setup

build.gradle.kts (Project) (Android only, versions as in your project)

```
plugins {
    id("com.google.dagger.hilt.android") version "2.52" apply false
    id("com.google.devtools.ksp") version "2.0.21-1.0.28" apply false
}
```

The app module's plugins and dependencies are in the list from Day 3. After changing Gradle files, click **Sync Now**.

3. The Hilt modules

ExpenseApp.kt and di/Modules.kt

```
@HiltAndroidApp
class ExpenseApp : Application()

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {
    @Provides
    @Singleton
    fun provideDatabase(@ApplicationContext context: Context): ExpenseDatabase =
        Room.databaseBuilder(context, ExpenseDatabase::class.java, "expenses.db").build()

    @Provides
```

```
    fun provideExpenseDao(db: ExpenseDatabase): ExpenseDao = db.expenseDao()
}

@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    @Singleton
    fun provideJson(): Json = Json { ignoreUnknownKeys = true }

    @Provides
    @Singleton
    fun provideRetrofit(json: Json): Retrofit = Retrofit.Builder()
        .baseUrl("https://api.frankfurter.app/")
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()

    @Provides
    @Singleton
    fun provideCurrencyApi(retrofit: Retrofit): CurrencyApi = retrofit.create(CurrencyApi::class.java)
}

@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
    @Binds
    @Singleton
    abstract fun bindExpenseRepository(impl: OfflineExpenseRepository): ExpenseRepository

    @Binds
    @Singleton
    abstract fun bindCurrencyRepository(impl: NetworkCurrencyRepository): CurrencyRepository
}

@Module
@InstallIn(SingletonComponent::class)
object TimeModule {
    @Provides
    fun provideTodayProvider(): TodayProvider = TodayProvider { LocalDate.now() }
}
```

Piece	Meaning
@HiltAndroidApp	Starts Hilt. Put it on your Application class (and register it in the manifest)
@InstallIn(SingletonComponent::class)	These objects live as long as the app
@Singleton	Create only one (one database, one Retrofit) and share it
@ApplicationContext	Hilt passes the app's Context, which Room needs
TodayProvider { LocalDate.now() }	Creates the fun interface from a lambda

AndroidManifest.xml: register the Application class

```
<application
    android:name=".ExpenseApp"
    android:label="@string/app_name"
    android:icon="@mipmap/ic_launcher"
    android:theme="@style/Theme.ExpenseTracker"
    ...>
    <activity android:name=".MainActivity" android:exported="true">
        ...
    </activity>
</application>
```

MainActivity.kt (Android only)

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            ExpenseTrackerTheme {
                ExpenseNavHost()      // Day 6 (and Week 16 adds the bottom bar)
            }
        }
    }
}
```

4. A ViewModel that gets everything injected

A first ViewModel to prove the wiring works

```
@HiltViewModel
class DebugViewModel @Inject constructor(
    repository: ExpenseRepository,
) : ViewModel() {
    val count: StateFlow<Int> = repository.observeExpenses()
        .map { it.size }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), 0)
}

@Composable
fun DebugScreen(viewModel: DebugViewModel = hiltViewModel()) {
    val count by viewModel.count.collectAsStateWithLifecycle()
    Text("Expenses in the database: $count", modifier = Modifier.padding(16.dp))
}
```

Put DebugScreen in setContent and run the app. If it shows "Expenses in the database: 0" without crashing, Hilt, Room and your repository are all connected. You can delete it after Day 5.

Error message	Usual cause
"... cannot be provided without an @Inject constructor or an @Provides-annotated method"	A class has no @Inject constructor, or an interface has no @Binds
"Hilt Activity must be attached to an @HiltAndroidApp Application"	android:name=".ExpenseApp" is missing from the manifest
"... is bound multiple times"	Two @Binds or @Provides for the same type
Crash: "Cannot create an instance of class ...ViewModel"	Missing @HiltViewModel, or @AndroidEntryPoint on the Activity

Practice Exercises

1. Write a **FakeTodayModule** (for your own experiments) that provides a TodayProvider that always returns 1 March 2025.
2. Add a **@Provides** function for an **HttpLoggingInterceptor** with level BODY, and an OkHttpClient that uses it. Then change provideRetrofit to take the OkHttpClient and call **.client(client)**.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. When do you use @Binds instead of @Provides?

Q2. Why is the database @Singleton?

Q3. What must be on MainActivity for hiltViewModel() to work?

Q4. Why can SaveExpenseUseCase be injected without any module?

WEEK 15 • DAY 5

The Expense List Screen

Today's goal: Build the home screen: expenses grouped by day, a monthly total, an empty state, and delete with undo.

1. Grouping expenses by day

The sketch shows a header for each day ("Today", "Yesterday", "Wed, 5 Mar") with that day's total. That's plain Kotlin logic, so write and run it first:

groupByDay() and dayLabel() (run)

```
data class DaySection(val date: LocalDate, val totalPaise: Long, val expenses: List<Expense>)

fun groupByDay(expenses: List<Expense>): List<DaySection> =
    expenses
        .groupBy { it.date }
        .toSortedMap(compareByDescending { it })
        .map { (date, list) -> DaySection(date, list.sumOf { it.amountPaise }, list) }

fun dayLabel(date: LocalDate, today: LocalDate): String = when (date) {
    today -> "Today"
    today.minusDays(1) -> "Yesterday"
    else -> date.format(java.time.format.DateTimeFormatter.ofPattern("EEE, d MMM"))
}

fun main() {
    val today = LocalDate.of(2025, 3, 10)
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, today),
        Expense(2, "Phone bill", 39900, Category.BILLS, today.minusDays(1)),
        Expense(3, "Auto", 28000, Category.TRAVEL, today),
        Expense(4, "Medicine", 15050, Category.HEALTH, LocalDate.of(2025, 3, 5)),
    )
    for (section in groupByDay(expenses)) {
        println("${dayLabel(section.date, today)} - ${formatRupees(section.totalPaise)}")
        section.expenses.forEach { println("   ${it.title}") }
    }
}
```

Output

```
Today - Rs. 400.00
   Lunch
   Auto
Yesterday - Rs. 399.00
   Phone bill
Wed, 5 Mar - Rs. 150.50
   Medicine
```

toSortedMap(compareByDescending { it }) sorts the groups newest first. (The real list from Room is already sorted, but sorting here too makes the function safe to use with any list.)

2. The UI state and the ViewModel

ExpenseListViewModel.kt (ui.list)

```
data class DaySection(val date: LocalDate, val totalPaise: Long, val expenses: List<Expense>)

data class ExpenseListUiState(
    val isLoading: Boolean = true,
    val sections: List<DaySection> = emptyList(),
    val monthTotalPaise: Long = 0,
)

fun groupByDay(expenses: List<Expense>): List<DaySection> =
    expenses
        .groupBy { it.date }
        .toSortedMap(compareByDescending { it })
        .map { (date, list) -> DaySection(date, list.sumOf { it.amountPaise }, list) }

@HiltViewModel
class ExpenseListViewModel @Inject constructor(
    private val repository: ExpenseRepository,
    private val todayProvider: TodayProvider,
) : ViewModel() {

    val uiState: StateFlow<ExpenseListUiState> = repository.observeExpenses()
        .map { list ->
            val thisMonth = YearMonth.from(todayProvider.today())
            ExpenseListUiState(
                isLoading = false,
                sections = groupByDay(list),
                monthTotalPaise = list.filter { YearMonth.from(it.date) == thisMonth }.sumOf { it.amountPaise },
            )
        }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), ExpenseListUiState())

    private var lastDeleted: Expense? = null

    fun delete(expense: Expense) {
        lastDeleted = expense
        viewModelScope.launch { repository.delete(expense.id) }
    }

    fun undoDelete() {
        val expense = lastDeleted ?: return
        lastDeleted = null
        viewModelScope.launch { repository.upsert(expense) }     // same id, so it comes back in place
    }
}
```

Piece	Why
isLoading = true at first	Until Room answers, the screen shows a spinner instead of "No expenses yet"
map { ... } then stateIn(...)	Turns the repository's Flow of expenses into a StateFlow of screen state (Week 9)
WhileSubscribed(5_000)	Stops listening to Room 5 seconds after the screen goes away, but survives rotation
lastDeleted + undoDelete()	Remembers the deleted expense, so Undo can save it again with the same id

3. The screen

As in Week 8, the screen is split in two: **ExpenseListScreen** talks to the ViewModel, and **ExpenseListContent** only takes state and callbacks, so it's easy to preview and test.

ExpenseListScreen.kt (ui.list)

```
@Composable
```

```
fun ExpenseListScreen(
    onAddClick: () -> Unit,
    onExpenseClick: (Long) -> Unit,
    viewModel: ExpenseListViewModel = hiltViewModel(),
) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()

    ExpenseListContent(
        state = state,
        snackbarHostState = snackbarHostState,
        onAddClick = onAddClick,
        onExpenseClick = onExpenseClick,
        onDelete = { expense ->
            viewModel.delete(expense)
            scope.launch {
                val result = snackbarHostState.showSnackbar(
                    message = "Deleted ${expense.title}",
                    actionLabel = "Undo",
                    duration = SnackbarDuration.Short,
                )
                if (result == SnackbarResult.ActionPerformed) viewModel.undoDelete()
            }
        },
    )
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ExpenseListContent(
    state: ExpenseListUiState,
    snackbarHostState: SnackbarHostState,
    onAddClick: () -> Unit,
    onExpenseClick: (Long) -> Unit,
    onDelete: (Expense) -> Unit,
    today: LocalDate = LocalDate.now(),
) {
    Scaffold(
        topBar = { TopAppBar(title = { Text("Expenses") }) },
        floatingActionButton = {
            FloatingActionButton(onClick = onAddClick) {
                Icon(Icons.Default.Add, contentDescription = "Add expense")
            }
        },
        snackbarHost = { SnackbarHost(snackbarHostState) },
    ) { padding ->
        when {
            state.isLoading -> Box(Modifier.fillMaxSize().padding(padding), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
            state.sections.isEmpty() -> EmptyState(Modifier.fillMaxSize().padding(padding))
            else -> LazyColumn(
                modifier = Modifier.fillMaxSize().padding(padding),
                contentPadding = PaddingValues(16.dp),
                verticalArrangement = Arrangement.spacedBy(8.dp),
            ) {
                item { MonthTotalCard(state.monthTotalPaise) }
                state.sections.forEach { section ->
                    item(key = "header-${section.date}") {
                        DayHeader(dayLabel(section.date, today), section.totalPaise)
                    }
                    items(section.expenses, key = { it.id }) { expense ->
                        ExpenseRow(expense, onClick = { onExpenseClick(expense.id) }, onDelete = { onDelete(expense) })
                    }
                }
            }
        }
    }
}
```

Tip: The snackbar is shown in **ExpenseListScreen**, not in the ViewModel. Showing messages is a UI job; the ViewModel only deletes and restores. **showSnackbar** is a suspend function that waits until the snackbar closes and tells you whether the user tapped Undo.

4. The small pieces

ExpenseListParts.kt (ui.list)

```
@Composable
fun MonthTotalCard(totalPaise: Long) {
    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {
            Text("This month", style = MaterialTheme.typography.labelLarge)
            Text(formatRupees(totalPaise), style = MaterialTheme.typography.headlineMedium)
        }
    }
}

@Composable
fun DayHeader(label: String, totalPaise: Long) {
    Row(Modifier.fillMaxWidth().padding(top = 8.dp)) {
        Text(label, style = MaterialTheme.typography.titleSmall, modifier = Modifier.weight(1f))
        Text(formatRupees(totalPaise), style = MaterialTheme.typography.titleSmall)
    }
}

@Composable
fun CategoryBadge(category: Category) {
    Box(
        modifier = Modifier.size(40.dp).clip(CircleShape).background(MaterialTheme.colorScheme.primaryContainer),
        contentAlignment = Alignment.Center,
    ) {
        Text(category.label.take(1), color = MaterialTheme.colorScheme.onPrimaryContainer)
    }
}

@Composable
fun ExpenseRow(expense: Expense, onClick: () -> Unit, onDelete: () -> Unit) {
    ListItem(
        modifier = Modifier.clip(RoundedCornerShape(12.dp)).clickable(onClick = onClick),
        leadingContent = { CategoryBadge(expense.category) },
        headlineContent = { Text(expense.title, maxLines = 1, overflow = TextOverflow.Ellipsis) },
        supportingContent = { Text(expense.category.label) },
        trailingContent = {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Text(formatRupees(expense.amountPaise), style = MaterialTheme.typography.titleMedium)
                IconButton(onClick = onDelete) {
                    Icon(Icons.Default.Delete, contentDescription = "Delete ${expense.title}")
                }
            }
        },
    )
}

@Composable
fun EmptyState(modifier: Modifier = Modifier) {
    Column(modifier.padding(32.dp), horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center) {
        Text("No expenses yet", style = MaterialTheme.typography.titleLarge)
        Spacer(Modifier.height(8.dp))
        Text("Tap + to add your first one.", textAlign = TextAlign.Center)
    }
}
```

CategoryBadge shows the first letter in a circle, so you don't need the large material-icons-extended library. Once the app works, you can switch to real icons if you like.

5. Preview it

A preview with sample data

```
@Preview(showBackground = true)
@Composable
fun ExpenseListPreview() {
    val today = LocalDate.of(2025, 3, 10)
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, today),
        Expense(2, "Auto", 28000, Category.TRAVEL, today),
        Expense(3, "Phone bill", 39900, Category.BILLS, today.minusDays(1)),
    )
    MaterialTheme {
        ExpenseListContent(
            state = ExpenseListUiState(isLoading = false, sections = groupByDay(expenses), monthTotalPaise = 79900),
            snackbarHostState = remember { SnackbarHostState() },
            onAddClick = {}, onExpenseClick = {}, onDelete = {},
            today = today,
        )
    }
}
```

Sketch: What the preview shows

```
+--------------------------------+
| Expenses                       |
|  This month                    |
|  Rs. 799.00                    |
|  Today              Rs. 400.00 |
|  (F) Lunch     Rs. 120.00  [x] |
|    Food                        |
|  (T) Auto      Rs. 280.00  [x] |
|  Yesterday          Rs. 399.00 |
|  (B) Phone bill Rs. 399.00 [x] |
|                           [ + ] |
+--------------------------------+
```

- Add a few expenses with a temporary button (or wait for tomorrow's form) and check they **stay after closing the app**.
- Rotate the phone: the list must not reload or lose anything.
- Delete an expense, then tap **Undo**: it should come back in the same place.

Practice Exercises

1. Add a **count** to DayHeader, so it shows "Today (2)". Change only DayHeader and the place that calls it.

2. Show the month total card in the **error colour** (MaterialTheme.colorScheme.error) when the total is more than Rs. 10,000. Write it as a parameter **isHigh: Boolean** of MonthTotalCard.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why does ExpenseListUiState start with isLoading = true?

Q2. Why is the snackbar shown from the screen and not the ViewModel?

Q3. What does key = { it.id } do in items()?

Q4. Why split ExpenseListScreen and ExpenseListContent?

WEEK 15 • DAY 6

Add and Edit Screen, and Navigation

Today's goal: Build one form for adding and editing, with validation errors, category chips, a date picker and navigation.

1. One screen for add and edit

Adding and editing need the same form. The only difference is the starting values. So there's one route with an **expenseId**: **0** means "add", any other number means "edit that expense".

Routes.kt

```
@Serializable object ExpenseListRoute
@Serializable data class AddEditRoute(val expenseId: Long = 0L)     // 0 = add a new expense
```

2. The form state and the ViewModel

AddEditViewModel.kt (ui.addedit)

```
data class AddEditUiState(
    val title: String = "",
    val amountText: String = "",
    val category: Category = Category.FOOD,
    val date: LocalDate = LocalDate.now(),
    val note: String = "",
    val errors: Map<Field, String> = emptyMap(),
    val isEditing: Boolean = false,
    val isSaved: Boolean = false,
)

@HiltViewModel
class AddEditViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val repository: ExpenseRepository,
    private val saveExpense: SaveExpenseUseCase,
    todayProvider: TodayProvider,
) : ViewModel() {

    private val expenseId = savedStateHandle.toRoute<AddEditRoute>().expenseId

    private val _uiState = MutableStateFlow(
        AddEditUiState(date = todayProvider.today(), isEditing = expenseId != 0L)
    )
    val uiState: StateFlow<AddEditUiState> = _uiState.asStateFlow()

    init {
        if (expenseId != 0L) {
            viewModelScope.launch {
                val e = repository.getExpense(expenseId) ?: return@launch
                _uiState.update {
                    it.copy(title = e.title, amountText = toAmountText(e.amountPaise),
                        category = e.category, date = e.date, note = e.note)
                }
            }
        }
    }

    // Each change also clears that field's error, so the red text goes away while typing.
    fun onTitleChange(v: String) = _uiState.update { it.copy(title = v, errors = it.errors - Field.TITLE) }
    fun onAmountChange(v: String) = _uiState.update { it.copy(amountText = v, errors = it.errors - Field.AMOUNT) }
    fun onCategoryChange(c: Category) = _uiState.update { it.copy(category = c) }
    fun onDateChange(d: LocalDate) = _uiState.update { it.copy(date = d, errors = it.errors - Field.DATE) }
    fun onNoteChange(v: String) = _uiState.update { it.copy(note = v) }

    fun save() {
```

```
        val s = _uiState.value
        viewModelScope.launch {
            when (val result = saveExpense(expenseId, ExpenseForm(s.title, s.amountText, s.category, s.date, s.note))) {
                ValidationResult.Valid -> _uiState.update { it.copy(isSaved = true) }
                is ValidationResult.Invalid -> _uiState.update { it.copy(errors = result.errors) }
            }
        }
    }
}
```

Piece	Why
savedStateHandle.toRoute<AddEditRoute>()	Reads the expenseId from the navigation route (Week 7)
amountText: String	The field keeps exactly what the user typed (like "99."); it's only parsed when saving
errors - Field.TITLE	Removes that field's error as soon as the user edits it
isSaved = true	Tells the screen to go back. The ViewModel never touches navigation itself
saveExpense(...)	All the rules live in the use case; the ViewModel just shows the result

3. The form screen

AddEditScreen.kt (ui.addedit)

```
@Composable
fun AddEditScreen(onDone: () -> Unit, viewModel: AddEditViewModel = hiltViewModel()) {
    val state by viewModel.uiState.collectAsStateWithLifecycle()

    // Saved? Go back. LaunchedEffect runs this once when isSaved becomes true.
    LaunchedEffect(state.isSaved) {
        if (state.isSaved) onDone()
    }

    AddEditContent(
        state = state,
        onTitleChange = viewModel::onTitleChange,
        onAmountChange = viewModel::onAmountChange,
        onCategoryChange = viewModel::onCategoryChange,
        onDateChange = viewModel::onDateChange,
        onNoteChange = viewModel::onNoteChange,
        onSave = viewModel::save,
        onBack = onDone,
    )
}

@OptIn(ExperimentalMaterial3Api::class, ExperimentalLayoutApi::class)
@Composable
fun AddEditContent(
    state: AddEditUiState,
    onTitleChange: (String) -> Unit,
    onAmountChange: (String) -> Unit,
    onCategoryChange: (Category) -> Unit,
    onDateChange: (LocalDate) -> Unit,
    onNoteChange: (String) -> Unit,
    onSave: () -> Unit,
    onBack: () -> Unit,
) {
    var showDatePicker by remember { mutableStateOf(false) }

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(if (state.isEditing) "Edit expense" else "New expense") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
```

```
            )
        },
    ) { padding ->
        Column(
            modifier = Modifier.padding(padding).padding(16.dp).verticalScroll(rememberScrollState()),
            verticalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            OutlinedTextField(
                value = state.title,
                onValueChange = onTitleChange,
                label = { Text("Title") },
                isError = Field.TITLE in state.errors,
                supportingText = { state.errors[Field.TITLE]?.let { Text(it) } },
                singleLine = true,
                modifier = Modifier.fillMaxWidth(),
            )
            OutlinedTextField(
                value = state.amountText,
                onValueChange = onAmountChange,
                label = { Text("Amount (Rs.)") },
                isError = Field.AMOUNT in state.errors,
                supportingText = { state.errors[Field.AMOUNT]?.let { Text(it) } },
                keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
                singleLine = true,
                modifier = Modifier.fillMaxWidth(),
            )

            Text("Category", style = MaterialTheme.typography.labelLarge)
            FlowRow(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                Category.entries.forEach { c ->
                    FilterChip(
                        selected = c == state.category,
                        onClick = { onCategoryChange(c) },
                        label = { Text(c.label) },
                    )
                }
            }

            OutlinedButton(onClick = { showDatePicker = true }, modifier = Modifier.fillMaxWidth()) {
                Text("Date: ${state.date}")
            }
            state.errors[Field.DATE]?.let { Text(it, color = MaterialTheme.colorScheme.error) }

            OutlinedTextField(
                value = state.note,
                onValueChange = onNoteChange,
                label = { Text("Note (optional)") },
                minLines = 2,
                modifier = Modifier.fillMaxWidth(),
            )

            Button(onClick = onSave, modifier = Modifier.fillMaxWidth()) {
                Text("Save")
            }
        }
    }

    if (showDatePicker) {
        // The date picker works with UTC milliseconds; one day = 86,400,000 ms.
        val pickerState = rememberDatePickerState(initialSelectedDateMillis = state.date.toEpochDay() * 86_400_000)
        DatePickerDialog(
            onDismissRequest = { showDatePicker = false },
            confirmButton = {
                TextButton(onClick = {
                    pickerState.selectedDateMillis?.let { onDateChange(LocalDate.ofEpochDay(it / 86_400_000)) }
                    showDatePicker = false
                }) { Text("OK") }
            },
            dismissButton = { TextButton(onClick = { showDatePicker = false }) { Text("Cancel") } },
        ) {
            DatePicker(state = pickerState)
        }
```

```
    }
}
```

UI piece	What to notice
isError + supportingText	The field turns red and shows the message from the errors map
KeyboardType.Decimal	Opens the number keyboard with a decimal point
FlowRow + FilterChip	Category chips that wrap to the next line on small screens
DatePickerDialog	Material 3's date picker. It uses UTC milliseconds, so we convert with 86,400,000 ms per day
verticalScroll	The form can scroll when the keyboard is open
LaunchedEffect(state.isSaved)	Runs onDone() once when saving succeeds

Careful: Don't call onDone() directly inside the composable body (**if (state.isSaved) onDone()** without LaunchedEffect). Composables can run many times, so navigation could happen twice. Side effects belong in LaunchedEffect.

4. Navigation

ExpenseNavHost.kt

```
@Composable
fun ExpenseNavHost() {
    val navController = rememberNavController()
    NavHost(navController, startDestination = ExpenseListRoute) {
        composable<ExpenseListRoute> {
            ExpenseListScreen(
                onAddClick = { navController.navigate(AddEditRoute()) },
                onExpenseClick = { id -> navController.navigate(AddEditRoute(id)) },
            )
        }
        composable<AddEditRoute> {
            AddEditScreen(onDone = { navController.popBackStack() })
        }
    }
}
```

Put **ExpenseNavHost()** in MainActivity's setContent, and delete the DebugScreen from Day 4. The app now does everything in the Must list: add, edit, delete with undo, a grouped list with a total, all saved in Room.

5. Test it like a user

Try this	Expected
Save with everything empty	Red errors under title and amount; nothing saved
Type "12.345" as the amount	"Enter a number like 120 or 99.50"
Pick tomorrow's date	"Date can't be in the future"
Tap an expense, change the amount, save	The list shows the new amount (same row, not a new one)
Fill the form, rotate the phone	Nothing typed is lost (it's in the ViewModel)
Close the app from recent apps and reopen it	All expenses are still there

Commit and tag the end of Week 15

```
git add .
git commit -m "Add expense list, add/edit form and navigation"
git push
git tag v0.1-mvp
git push --tags
```

Tip: Ask a teammate (or your mentor) to try the app for five minutes without your help. Watch where they get stuck. That's exactly what real teams do before a release.

Practice Exercises

1. Add a **Delete** icon button to the top bar of the edit screen (only when **isEditing**). It should call a new **delete()** function in AddEditViewModel that deletes the expense and sets isSaved = true.

2. Show the date on the button as "Mon, 10 Mar 2025" instead of "2025-03-10". Write a small function **formatLongDate(date)** using DateTimeFormatter.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. How does the same screen know if it's adding or editing?

Q2. Why is amountText a String in the UI state?

Q3. Why use LaunchedEffect for navigating back after saving?

Q4. Where do the validation rules live?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Plan Your Final Project

Exercise 1

```
import java.time.LocalDate

enum class Category(val label: String) { FOOD("Food"), TRAVEL("Travel"), BILLS("Bills") }
enum class PaymentMethod(val label: String) { CASH("Cash"), UPI("UPI"), CARD("Card") }

data class Expense(
    val id: Long = 0,
    val title: String,
    val amountPaise: Long,
    val category: Category,
    val date: LocalDate,
    val paymentMethod: PaymentMethod = PaymentMethod.UPI,
)

fun main() {
    val tea = Expense(1, "Tea", 1500, Category.FOOD, LocalDate.of(2025, 3, 10), PaymentMethod.CASH)
    println(tea)
}
```

Output

```
Expense(id=1, title=Tea, amountPaise=1500, category=FOOD, date=2025-03-10, paymentMethod=CASH)
```

Exercise 2

```
fun List<Expense>.totalFor(category: Category): Long =
    filter { it.category == category }.sumOf { it.amountPaise }

fun main() {
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, LocalDate.of(2025, 3, 3)),
        Expense(2, "Bus pass", 50000, Category.TRAVEL, LocalDate.of(2025, 3, 1)),
        Expense(3, "Dinner", 28000, Category.FOOD, LocalDate.of(2025, 3, 8)),
    )
    println("Food: ${expenses.totalFor(Category.FOOD)} paise")
}
```

Output

```
Food: 40000 paise
```

Quiz answers

Q1. Minimum Viable Product: the smallest useful version. You get a working app early and add extras later.

Q2. Only valid values are possible, so there are no typos, and when() can check every case.

Q3. 0 means "not saved yet"; Room generates the real id when the expense is inserted.

Q4. API 26.

Day 2: The Domain Layer: Money, Rules and Use Cases

Exercise 1

```
enum class Field { TITLE, AMOUNT, DATE, NOTE }

data class ExpenseForm(val title: String, val amountText: String, val category: Category,
                       val date: LocalDate, val note: String = "")

fun validate(form: ExpenseForm, today: LocalDate): Map<Field, String> {
    val errors = mutableMapOf<Field, String>()
    if (form.title.isBlank()) errors[Field.TITLE] = "Enter a title"
    val paise = parseRupees(form.amountText)
    if (paise == null || paise <= 0) errors[Field.AMOUNT] = "Enter a valid amount"
    if (form.date.isAfter(today)) errors[Field.DATE] = "Date can't be in the future"
    if (form.note.trim().length > 100) errors[Field.NOTE] = "Note is too long"     // the new rule
    return errors
}

fun main() {
    val today = LocalDate.of(2025, 3, 10)
    println(validate(ExpenseForm("Books", "450", Category.OTHER, today, "x".repeat(120)), today))
    println(validate(ExpenseForm("Books", "450", Category.OTHER, today, "For the exam"), today))
}
```

Output

```
{NOTE=Note is too long}
{}
```

Exercise 2

```
fun averagePerDay(expenses: List<Expense>, month: YearMonth): Long =
    summarize(expenses, month).totalPaise / month.lengthOfMonth()

fun main() {
    val expenses = listOf(
        Expense(1, "Lunch", 12000, Category.FOOD, LocalDate.of(2025, 3, 3)),
        Expense(2, "Bus pass", 50000, Category.TRAVEL, LocalDate.of(2025, 3, 1)),
        Expense(3, "Dinner", 28000, Category.FOOD, LocalDate.of(2025, 3, 8)),
        Expense(4, "Phone bill", 39900, Category.BILLS, LocalDate.of(2025, 3, 5)),
    )
    println("Average per day: " + formatRupees(averagePerDay(expenses, YearMonth.of(2025, 3))))
}
```

Output

```
Average per day: Rs. 41.90
```

Quiz answers

Q1. Double can't store numbers like 0.1 exactly, so totals drift. Whole paise in a Long are always exact.

Q2. null, because money can't have more than 2 decimal places.

Q3. So tests can choose the date; the result doesn't depend on when the test runs.

Q4. Nothing Android: only plain Kotlin (and Flow). The data and UI layers depend on it.

Day 3: The Data Layer: Room and Retrofit

Exercise 1

```
@Query("SELECT * FROM expenses WHERE epoch_day BETWEEN :startDay AND :endDay ORDER BY epoch_day DESC, id DESC")
fun observeBetween(startDay: Long, endDay: Long): Flow<List<ExpenseEntity>>

// Example: all of March 2025
// dao.observeBetween(LocalDate.of(2025, 3, 1).toEpochDay(), LocalDate.of(2025, 3, 31).toEpochDay())
```

Exercise 2

```
import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json

@Serializable
data class RatesDto(val amount: Double, val base: String, val date: String, val rates: Map<String, Double>)

fun RatesDto.convert(paise: Long, to: String): Double? {
    val rate = rates[to] ?: return null
    return paise / 100.0 * rate
}

fun main() {
    val json = """{"amount":1.0,"base":"INR","date":"2025-03-07","rates":{"USD":0.01148}}"""
    val dto = Json.decodeFromString<RatesDto>(json)
    println(dto.convert(1_250_000, "USD"))
    println(dto.convert(1_250_000, "EUR"))
}
```

Output

```
143.5
null
```

Quiz answers

Q1. It keeps database details (column names, types, annotations) out of the domain layer. Each layer can change on its own.

Q2. As a Long: the number of days since 1 January 1970 (toEpochDay).

Q3. -1. That's why the repository returns the expense's own id in that case.

Q4. The network can fail. Result lets the caller handle success and failure without the app crashing.

Day 4: Wiring Everything with Hilt

Exercise 1

```
@Module
@InstallIn(SingletonComponent::class)
object FakeTodayModule {
    @Provides
    fun provideTodayProvider(): TodayProvider = TodayProvider { LocalDate.of(2025, 3, 1) }
}
// Use it instead of TimeModule (not both, or Hilt reports "bound multiple times").
```

Exercise 2

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    @Singleton
    fun provideLogging(): HttpLoggingInterceptor =
        HttpLoggingInterceptor().apply { level = HttpLoggingInterceptor.Level.BODY }

    @Provides
    @Singleton
    fun provideOkHttp(logging: HttpLoggingInterceptor): OkHttpClient =
        OkHttpClient.Builder().addInterceptor(logging).build()

    @Provides
    @Singleton
    fun provideJson(): Json = Json { ignoreUnknownKeys = true }

    @Provides
    @Singleton
    fun provideRetrofit(json: Json, client: OkHttpClient): Retrofit = Retrofit.Builder()
        .baseUrl("https://api.frankfurter.app/")
        .client(client)
        .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
        .build()
}
```

Quiz answers

Q1. To tell Hilt which class to use for an interface, when that class already has an @Inject constructor.

Q2. Opening a database is slow, and two instances can cause bugs; the whole app should share one.

Q3. @AndroidEntryPoint.

Q4. It has an @Inject constructor, and Hilt knows how to make all its parameters.

Day 5: The Expense List Screen

Exercise 1

```
@Composable
fun DayHeader(label: String, count: Int, totalPaise: Long) {
    Row(Modifier.fillMaxWidth().padding(top = 8.dp)) {
        Text("$label ($count)", style = MaterialTheme.typography.titleSmall, modifier = Modifier.weight(1f))
        Text(formatRupees(totalPaise), style = MaterialTheme.typography.titleSmall)
    }
}

// In ExpenseListContent:
// DayHeader(dayLabel(section.date, today), section.expenses.size, section.totalPaise)
```

Exercise 2

```
@Composable
fun MonthTotalCard(totalPaise: Long, isHigh: Boolean) {
    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {
            Text("This month", style = MaterialTheme.typography.labelLarge)
            Text(
                formatRupees(totalPaise),
                style = MaterialTheme.typography.headlineMedium,
                color = if (isHigh) MaterialTheme.colorScheme.error else MaterialTheme.colorScheme.onSurface,
            )
        }
    }
}

// In ExpenseListContent:
// item { MonthTotalCard(state.monthTotalPaise, isHigh = state.monthTotalPaise > 1_000_000) }
```

Quiz answers

Q1. So the screen shows a spinner until the first data arrives, instead of wrongly showing the empty state.

Q2. Showing messages is UI work. The ViewModel only changes data (delete/undo).

Q3. It gives each row a stable identity, so Compose keeps the right state and animates changes correctly.

Q4. Content has no ViewModel, so it can be previewed and tested with any state.

Day 6: Add and Edit Screen, and Navigation

Exercise 1

```
// In AddEditViewModel:
fun delete() {
    viewModelScope.launch {
        repository.delete(expenseId)
        _uiState.update { it.copy(isSaved = true) }
    }
}

// In AddEditContent (add a parameter onDelete: () -> Unit), inside TopAppBar:
// actions = {
//     if (state.isEditing) {
//         IconButton(onClick = onDelete) { Icon(Icons.Default.Delete, contentDescription = "Delete expense") }
//     }
// }
```

Exercise 2

```
fun formatLongDate(date: LocalDate): String =
    date.format(java.time.format.DateTimeFormatter.ofPattern("EEE, d MMM yyyy"))

// In AddEditContent:
// Text("Date: ${formatLongDate(state.date)}")
```

Quiz answers

Q1. From the route's expenseId: 0 means add; any other id means load that expense and edit it.

Q2. So the field shows exactly what the user typed, even half-finished input like "99."; it's validated only when saving.

Q3. Composables can run many times. LaunchedEffect runs the side effect once per key change.

Q4. In the domain layer (validate() inside SaveExpenseUseCase), not in the screen or the ViewModel.

Next: When you finish this week, tell your mentor and ask for the **Week 16** material. Great work, Anjana!